Agentic Engineering

with Claude Code

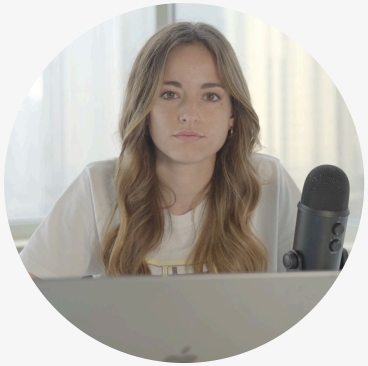
Elena Hernández-Martínez
Kristof Schröder
Tim Mensinger

PyData Masterclass

appliedAI institute for europe
A joint initiative
UNTERNEHMER TUM
IPAI

Welcome and Introduction

Meet the trainers

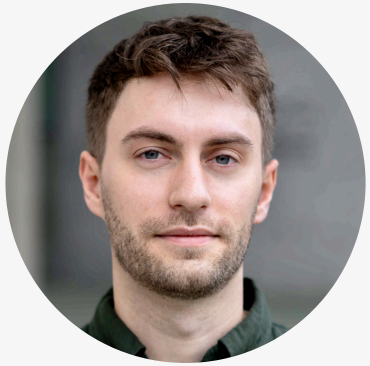


Elena

AI Researcher

Kristof

Principal Research Engineer



Tim

AI Researcher

How to get the slides

https://aai-training-agentic-engineering.netlify.app

Password: ae@aai

Agenda

Morning

- Introduction
- Getting Started
- Prompt & Context Engineering
- Customization
- Permissions & Sandboxing

Afternoon

- Ownership & Understanding
- Skills
- MCP & Subagents
- Workflows
- Closing Remarks

Coffee and Food?

- Coffee breaks (~20-30 minutes)
  - around 10:00
  - around 14:45
- Lunch
  - at 12:00

How each section works

1. Presentation

We introduce the topic and key concepts

→

2. Exercise

You apply the concepts hands-on with Claude Code

→

3. Discussion

We talk through solutions and share observations

How do we want to work together?

Ask Questions!

Interrupt us anytime, questions help everyone learn

Be Open!

Share observations and learn from each other

Anything else?

Is there something we should know or pay attention to?

appliedAI Institute for Europe gGmbH

UnternehmerTUM

The largest entrepreneurship and innovation center in Europe

IPAI (Innovationspark KI Heilbronn)

The largest lighthouse project for an application ecosystem of AI in Europe

initiative for applied artificial intelligence

appliedAI is a joint venture between Europe's strongest ecosystems for innovation and AI.

appliedAI institute for europe

appliedAI Institute is the **open-access accelerator for trustworthy AI.**

The appliedAI Institute for Europe is the open-access accelerator for Trustworthy AI

Our Mission

Empowering professionals to develop and apply latest AI technologies responsibly, shaping a future that we desire to live in.

The appliedAI Institute for Europe has following focus areas:

- **AI Development & AI Adoption**
- **AI Skills & AI Literacy**
- **AI for Social, Public & Ecological Impact**
- **AI Regulation & AI Governance**

The appliedAI Institute wants to fulfill the mission within these four areas by creating:

Knowledge

Empower professionals from industry, research & public sector through high-quality knowledge

Resources

Empowering professionals by using open-source tools and free resources

Community

Supporting professionals by bringing together the community & share knowledge and experience

For more information click here

Disclaimer

Disclaimer:

- **No legal advice.**

  Content is for educational purposes only.

- **AI can break things.**

  Outputs may be wrong and can modify or delete data. The organizers assume no liability.

- **Be aware of using sensitive or personal data.**

  If you use it anyway. You carry full responsibility.

Warmup Game

Quick Warmup (1/2)

How often do you use coding agents?

- Never
- Sometimes
- Often
- Always

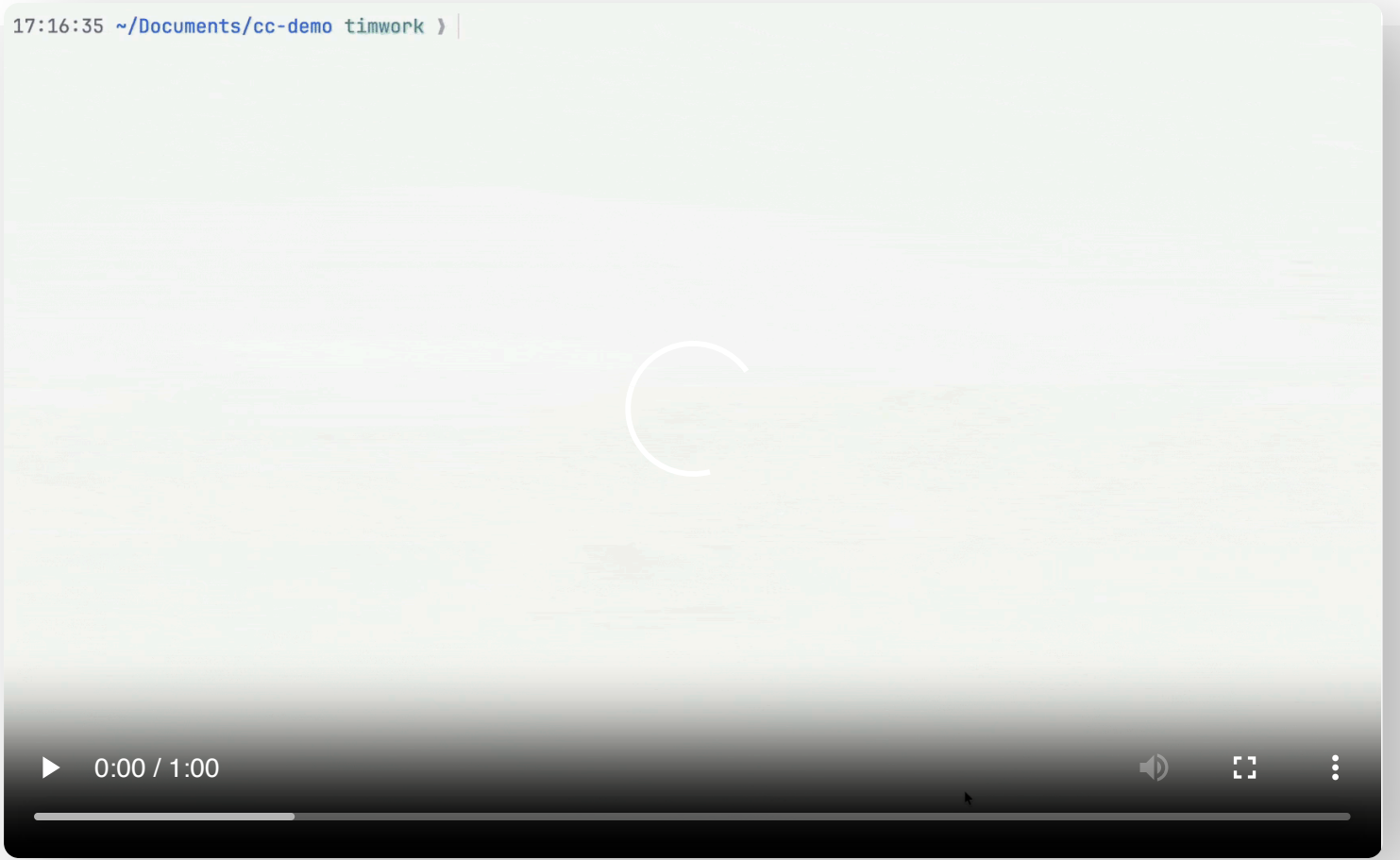
Quick Warmup (2/2)

How many of these do you know?

CLAUDE.md	MCP	Agentic loop	Subagents
Context engineering	Prompt engineering	Hooks	Skills
Permission system	Sandboxing	RPI Workflow	Rules

Getting Started

Claude Code in action



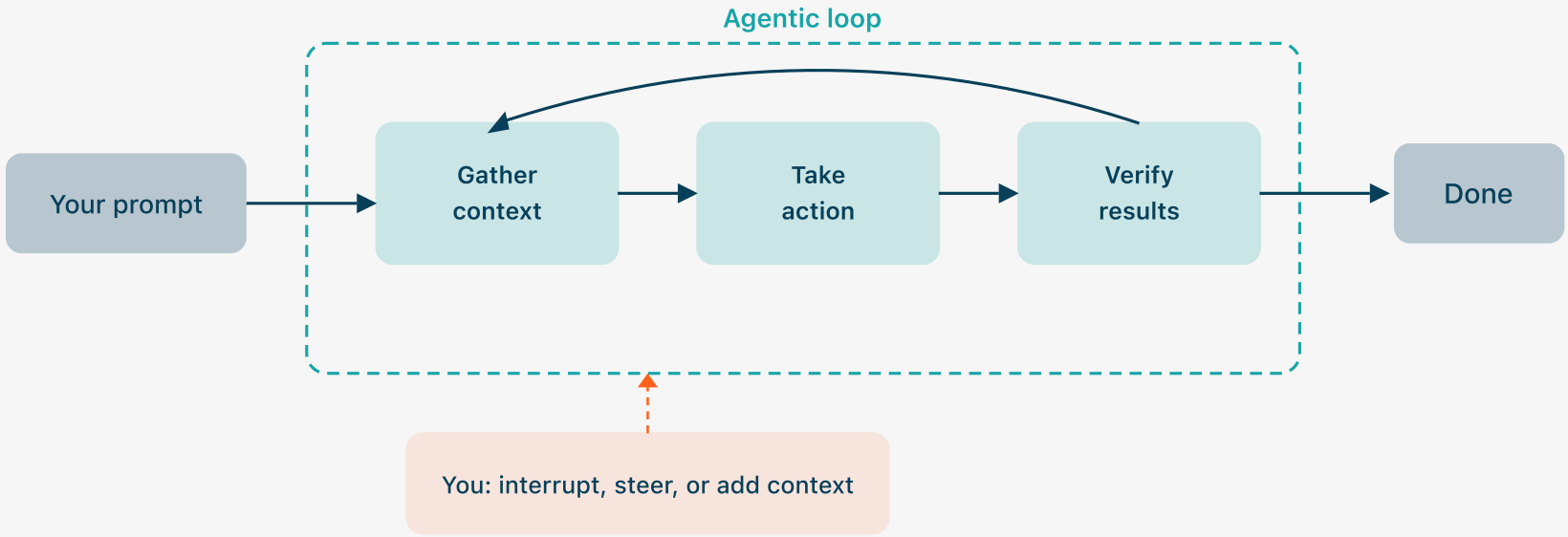
An alien tool with no manual

> *Clearly some powerful alien tool was handed around except it comes with no manual and everyone has to figure out how to hold it and operate it.*
>
> — Andrej Karpathy, December 2025

Who recognizes this feeling?

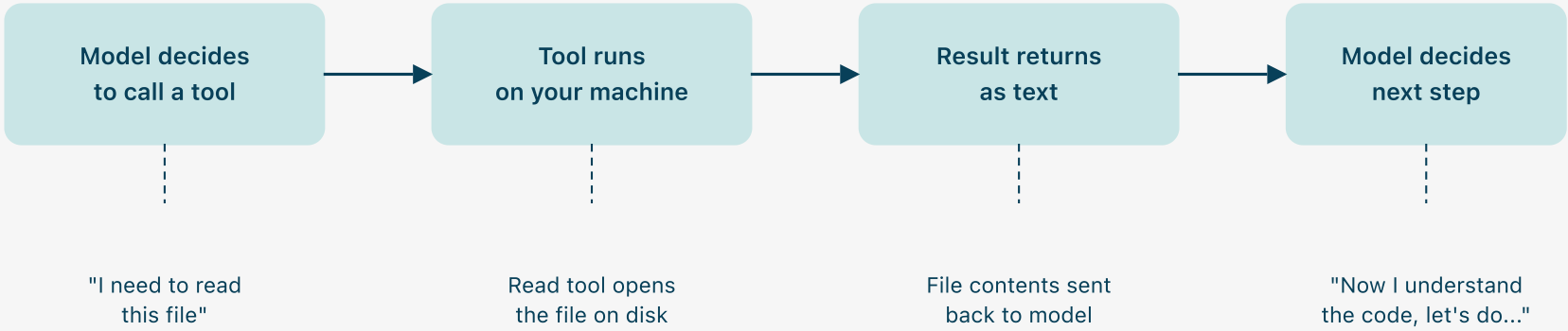
Coding agents basics

The agentic loop



How the agent acts: Tools

In each phase of the agentic loop, the model acts through **tool calls**.



The bottleneck moved

Coding with agents changes what's hard.

Writing code is no longer the bottleneck.

Planning, steering, and reviewing are.

It's easy to lose control

> *You can systematically refactor, but invariably an agentic codebase will end up larger and more overwrought than anything built by human hand. This is technical debt on an unprecedented scale, accrued at machine speed.*
>
> — Wes McKinney

> *If you don't understand the [AI-generated] code, your only recourse is to ask AI to fix it for you, which is like paying off credit card debt with another credit card.*
>
> — Steve Krouse

The rest of this course is about keeping control.

~~*"...some powerful alien tool with no manual..."*~~

Stuck? Ask Claude Code.

When you ask Claude Code about itself, it queries its own online documentation.

Cheatsheet

Input

`Enter`	Submit your prompt
`Shift+Enter` (or: `\` then `Enter`)	Multiline input (Run `/terminal-setup` to enable `Shift+Enter`)

Control

`Esc` (or: `Ctrl+C`)	Stop Claude mid-response
`/exit` (or: `Ctrl+D`)	Exit Claude Code
`Up/Down` arrows	Navigate command history
`Left/Right` arrows	Cycle through dialog tabs

How to get the slides

https://aai-training-agentic-engineering.netlify.app

Password: ae@aai

How to get the exercises

1. Clone the exercise repository

```
git clone https://github.com/aai-institute/training-agentic-engineering-exercises.git
```

2. Navigate into the exercise directory and open Claude Code

```
cd training-agentic-engineering-exercises/01-getting-started
claude
```

3. Open the `README.md` for the exercise description.

Exercises: Getting Started

Duration: 20 minutes

Prompt and Context Engineering

Definitions

Prompt Engineering

Communicating your intent clearly: what you want, why, and what "done" looks like.

Context Engineering

Curating the information the agent sees so it can act correctly.

Just talk to it

> *Just talk to it. Play with it. Develop intuition.*
>
> — Peter Steinberger (OpenClaw)

- Classic prompt engineering techniques (role assignment) show diminishing returns on modern models [1]
- Models have gotten dramatically better at instruction following. You don't need tricks.
- **One-liners are often enough.** Iteration is a legitimate alternative to the perfect prompt.

[1] Anthropic, "Effective Context Engineering" (2025): "Building with language models is becoming less about finding the right words..."

When a one-liner is not enough

If Claude Code can’t solve a problem, or takes much longer than you have anticipated, consider adding the following to your prompt:

- **Constraints and boundaries**: files to touch, APIs to use, conventions to follow
- **A clear definition of done**: tests passing, a specific behavior
- **Examples or references**: point to existing code, a pattern, or a doc to follow

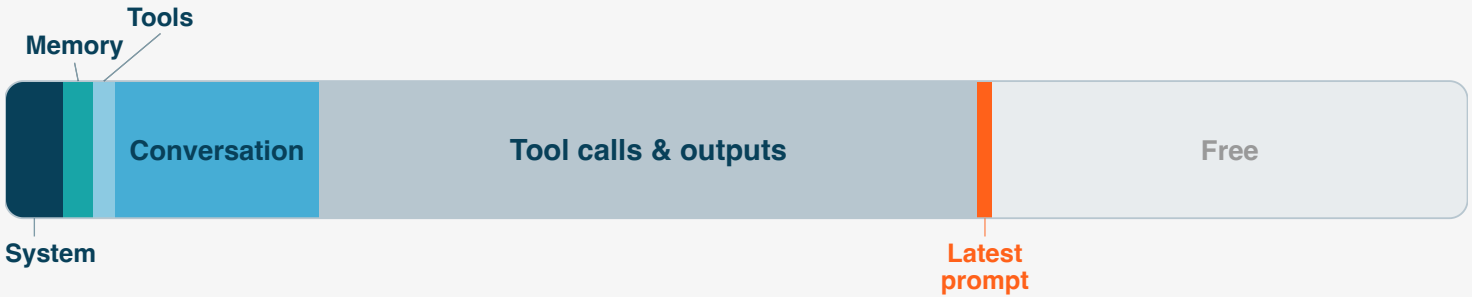
→ Good prompt engineering is knowing when to move from simple to detailed.

→ You use coding agents to save time; don’t spend it crafting complex prompts for simple tasks.

> 💡 **TIP**
>
> `Ctrl+G` opens your prompt in a text editor (configurable via `EDITOR` environment variable).

What Claude Code actually sees

Illustration: Context window (mid-session)



Your prompt is a small part of what Claude sees. Everything in context **competes for attention**.

→ Context must be managed well!

> 💡 **TIP**
>
> Run `/context` to see the context breakdown of your conversation. The labels will differ from this illustration.

Context is finite

Claude Code uses a **1M token** context window (Opus 4.6).

`pytest` (~370K tokens) → **37%** of the window

`pandas` (~2.5M tokens) → **does not fit**

- Context is shared with everything from the previous slide: system prompt, conversation history, tool outputs, etc.
- Even if information fits inside the context window, the model might not be able to use it well.

Context degradation

As conversations grow longer quality degrades (e.g., forgets a constraints). **Two effects stand out:**

Position: the middle gets lost

- Models attend most to the beginning and end of context, least to the middle (U-shaped curve).
- Baked into the transformer architecture ("Lost in the Middle," Liu et al., 2024).

Length: more tokens, worse retrieval

- Needle in the Haystack benchmarks: Hide a fact in a long context, ask the model to retrieve it.
- All frontier models degrade (Chroma, 2025; MRCR v2, 2026).

- But it is not only position or length: Irrelevant or incorrect context lowers the **signal-to-noise ratio**.
- All of this is referred to as **context rot**.

  → **Avoiding context rot is the single most important skill for coding agent users.**

Avoiding context rot

> *Find the smallest possible set of high-signal tokens that maximize the likelihood of your desired outcome.*
>
> — Anthropic, 2025 [1]

Prevention

- **One task per conversation.** `/clear` between topics.
- **Check usage** with `/context`.
- **Stage:** explore → plan → `/clear` → implement.
- **Isolate side quests** in subagents or parallel sessions.

Mitigation

- **Compress** context manually using **`/compact`**.
- **Undo** a wrong turn before it pollutes further context with **`/rewind`**.
- **Reset** and start fresh using **`/clear`**.

[1] Anthropic, "Effective Context Engineering" (2025)

How /rewind works

Every prompt and write operation creates a checkpoint.

From the menu, pick a checkpoint and choose:

- **Restore code + conversation** — full undo to that point
- **Restore code only** — revert files, keep the conversation
- **Restore conversation only** — rewind the chat, keep current code
- **Summarize from here** — compress messages from that point forward (targeted compaction)

> ⚠ **WARNING**
>
> Only tracks Claude's file edits. Bash side effects or manual edits are not checkpointed.

How /compact works

1. Claude **summarizes** the conversation into a short block
2. All messages before the summary are **dropped**
3. System prompt and CLAUDE.md are **re-injected**

Add focus instructions to control what's preserved using:

`/compact [focus]`

Auto-compact triggers near the context limit, but you have no control over what it preserves. If you want control, try to avoid this!

> ⚠ **WARNING**
>
> Exact wording, intermediate details, and anything not captured in the summary are lost.

Cheat sheet

appliedAI institute for europe

Input

`@file`	Pull a file's content into context (with autocomplete)
`! [shell command]`	Run a shell command and add its output to context

Inspect context

`/context`	Visualize context usage

Manage context

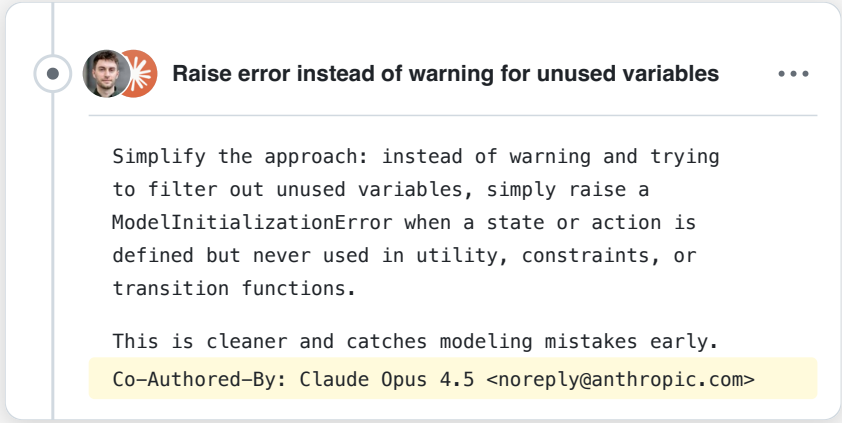
`/compact [focus]`	Replace conversation with a summary, freeing context
`/clear`	Reset conversation history entirely
`/rewind`	Restore code and/or conversation to a previous point
`/btw <question>`	Side question that never enters history
`/resume`	Resume a previous session
`/fork`	Branch the current conversation

Exercises: Prompt and Context Engineering

Duration: 20 minutes

Customization

Seen this before?



"I want to tell Claude to stop doing this." → **CLAUDE.md**

"Can Claude learn my preferences?" → **Auto-memory**

"I need a guarantee it happens every time." → **Hooks**

"Is there an official toggle?" → **Settings**

A README for your agent

README files tell other *humans* why a project is useful, what they can do with it, and how they can use it.

In a README file for *agents*, you capture precise and agent-focused instructions that facilitates the collaboration of the developer and the agent on a project.

- Instructions are stored as text in a `CLAUDE.md` file
- `CLAUDE.md` is loaded into **every** conversation at session start
- Other coding agents use `AGENTS.md` instead of `CLAUDE.md`

  Write `@AGENTS.md` inside `CLAUDE.md` to import without duplication

CLAUDE.md

```
- Do not add attributions to commit
  messages

- ...
```

> 💡 **TIP**
>
> Use `/init` to generate a CLAUDE.md from your repo. It's a starting point, not the final version.

There is no correct level of detail

CLAUDE.md

```
you are on WSL on Windows
```

CLAUDE.md

```
- Use `uv` for package management
- Use `ruff` for formatting
- Use `just -l` to find all available
  recipes
- Use `polars` instead of `pandas`
```

CLAUDE.md

```
# Writing Code
Simple, clean, maintainable over
clever or complex. Match existing
file styling.

## Autonomy Levels
- 🟢 Fix tests, linting, typos
- 🟡 Multi-file changes, new features
- 🔴 Rewriting working code, security

# Git
Never use `--no-verify`.
Read pre-commit errors aloud before
fixing. Identify root cause first.

# Testing
TDD: write tests first. All projects
need unit, integration, AND e2e tests.
```

Writing instructions

Do's

- **Don't write all instructions at once!** Add one instruction at a time when you recognize the need.
- **Keep it short.** Aim for under ~100 instructions. Compliance degrades as the count grows.[1]
- **Explain *why*, not just *what*.** Claude can judge edge cases when it knows the why, not just the what.

Don'ts

- **Don't add info that is easily discoverable.** Claude reads your repo, it can find scripts, frameworks, and structure on its own.
- **Don't send an LLM to do a linter's job.** Style enforcement belongs in formatters and hooks.
- **Don't trust /init blindly.** Good starting point, but requires (heavy) pruning.[2]

[1]How Many Instructions Can LLMs Follow at Once? (Jaroslawicz et al., 2025).
[2]Evaluating AGENTS.md (Gloaguen et al., 2026).

Scopes

Scope	Location	Shared with	Example
User	`~/.claude/CLAUDE.md`	Just you (all projects)	*"Never use emojis in commit messages"*
Project	`.claude/CLAUDE.md`	Team via source control	*"We use `uv` for dependency management"*
Project-local	`.claude/CLAUDE.local.md`	Just you (this project)	*"I'm new to this code, explain in detail"*
Subfolder	`src/rust-core/CLAUDE.md`	Team via source control	*"Use `cargo clippy` before committing"*

Subfolder instructions are only injected into the context if Claude accesses the subfolder.

⚠ **WARNING**

- All scopes are **concatenated** into one context, not merged!
- There is no override mechanism, so contradictory instructions across scopes can cause unpredictable behavior.

→ Unsure? Ask Claude to check whether your instructions conflict!

Modularizing instructions

Subfolder `CLAUDE.md` files scope by directory. **Rule files** let you modularize by topic.

```
.claude/
├── CLAUDE.md
└── rules/
    ├── attributions.md
    └── formatting.md
```

`.claude/rules/attributions.md`

```
Do not add attributions to commit
messages.
```

Loaded at session start, same as `CLAUDE.md`.

`.claude/rules/formatting.md`

```
---
paths:
  - "**/*.py"
---
Format all Python files with ruff
after editing.
```

Loaded only when Claude reads files matching the path.

> ℹ️ **NOTE**
>
> Without path-scoping, rule files don't save context. The split primarily helps the developers, not the model.

Auto-memory

- With CLAUDE.md and rules, **you** decide what the agent should know.
- Auto-memory is a feature where Claude decides **on its own** which preference to store.

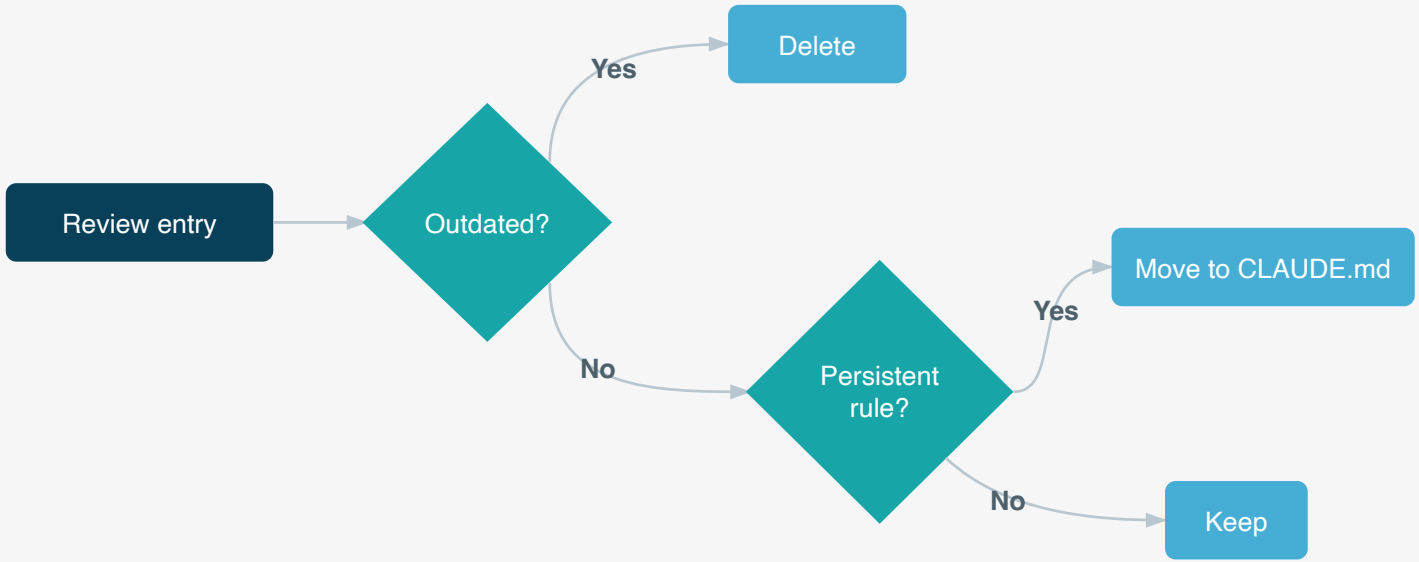
Enabled by default. Claude saves notes about your preferences to machine-local markdown files.

Inspect and manage with `/memory`. You can deactivate auto-memory here.

Over time, memories can **conflict with your instructions** or go stale. Curate regularly.

Auto-dream (off by default) consolidates memories between sessions: prunes stale entries, merges duplicates, resolves contradictions.

Uncurated memory is worse than no memory!



→ Either curate regularly or deactivate.

The limits of asking nicely

CLAUDE.md

```
Format Python files with ruff
after editing.
```

CLAUDE.md

```
ALWAYS format Python files with
ruff after editing. No exceptions!
```

CLAUDE.md

```
🚨 IMPORTANT: You MUST run ruff
format on EVERY Python file after
editing. DO NOT SKIP THIS!!! 😠
```

But this is a deterministic task, can't we just run `ruff` after every Write-tool call?

→ Yes, and this is what **Hooks** allow you to do!

Hooks

When – The Event

The hook should fire after Claude uses a tool

→ `"PostToolUse"`

Which – The Matcher

The hook should fire only when the Edit or Write tool was used.

→ `"matcher": Edit|Write`

What – The Handler

The hook runs `uvx ruff format .` to format all Python files.

`.claude/settings.json`

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "uvx ruff format ."
          }
        ]
  ...
}
```

What else can hooks do?

Did Claude actually finish the task?

Instruct a fast LLM with a prompt after Claude stops to verify that the task was actually solved.

- **Event:** `Stop`
- **Handler:** `prompt`

Get notified by Claude

Let Claude sent you a desktop notification if it requires your input or is finished.

- **Event:** `Notification`
- **Handler:** `command`

Block dangerous commands

Block dangerous commands before execution using a determinstic logic (or using another LLM as judge).

- **Event:** `PreToolUse`
- **Handler:** `command` (or `prompt`)

...

Before you go build hooks

There's much more.

As of right now, there are 26 event types and 4 handler types. Check them out on Anthropic's official hooks guide.

Same scoping as instructions files

- Personal: `~/.claude/settings.json`
- Project-shared: `.claude/settings.json`
- Project-personal: `.claude/settings.local.json`

⚠ WARNING

Hooks can run arbitrary code!

- Claude Code does not ask for permission before executing hook-scripts
- Be extra careful when using hooks from others
- When opening Claude in a new project, read through the existing hooks before trusting the folder!

Choosing the right mechanism

Need	Reach for	Why
"Try to avoid very deep functions"	**CLAUDE.md**	Not deterministic and requires judgment
"We use uv"	**CLAUDE.md**	Context to increase Claude's efficiency
Format Python with ruff after every edit	**Hook**	Deterministic, no LLM needed
Validate Bash commands with an LLM before running	**Hook**	Custom logic beyond built-in permissions
Don't let Claude read files in `data/`	**Permission** (Sec. 4)	Intuitive built-in access control
Go through a PR review checklist	**Skill** (Sec. 5)	Want manual invocation

Some other settings

Attribution

Remember the Co-Authored-By line? Toggle it off for commits and PR descriptions.

Status line

Always-visible bar with live session info. Configure with `/statusline`.

Spinner verbs

Replace "Thinking..." with your own messages.

`.claude/settings.json`

```
{
  "attribution": {
    "commits": false,
    "pullRequests": false
  },
  "statusLine": {
    "type": "command",
    "command": "~/.claude/statusline.sh"
  },
  "spinnerVerbs": {
    "mode": "replace",
    "verbs": [
      "Brewing coffee",
      "Massaging the tensors",
      "Consulting the oracle"
    ]
  },
  ...
}
```

Cheat sheet

appliedAI institute for europe

Customization files

`.claude/CLAUDE.md`	Instructions (project, shared)
`.claude/CLAUDE.local.md`	Instructions (project, personal)
`~/.claude/CLAUDE.md`	Instructions (user, all projects)
`.claude/rules/*.md`	Modular rules (path-scoped optional)
`~/.claude/projects/.../memory/`	Auto-memory (machine-local)
`~/.claude/settings.json`	Hooks and other settings (user, all projects)
`.claude/settings.json`	Hooks and other settings (project, shared)
`.claude/settings.local.json`	Hooks and other settings (project, personal)

Commands

`/init`	Generate CLAUDE.md for the project folder
`/memory`	View and manage auto-memory and instructions
`/hooks`	Inspect current hook configuration
`/statusline`	Configure the status line

Exercises: Customization

Duration: 20 minutes

Permissions and Sandboxing

A real-world example

> *I over-relied on the AI agent to run Terraform commands. I treated `plan`, `apply`, and `destroy` as something that could be delegated. ... Agents no longer execute commands. Every plan is reviewed manually. Every destructive action is run by me.*
>
> — Alexey Grigorev, March 2026

Asked Claude Code to clean up duplicate Terraform resources. The agent ran `terraform destroy`.

2.5 years of production data gone.

Claude Code's security model

Your review and judgement

The primary safeguard. The agent proposes, you decide. No technical control replaces this.

Permission-based architecture

Controls what the agent is allowed to do: which tools require approval, which are auto-approved, which are denied

Built-in protections

Natural language command descriptions, prompt injection safeguards, sandboxing (opt-in)

⚠ WARNING

You are part of the security model. The agent only acts within the permissions you grant it. Reviewing what you approve is your responsibility.

Tool categories and default behavior

Read-only

`Read`, `Grep`, `Glob`

Auto-approved

File modification

`Edit`, `Write`

Requires approval

Bash commands

Shell execution

Requires approval

Web

`WebFetch`, `WebSearch`

Requires approval

More tools

See the full list in the **tools reference →**

> ℹ️ NOTE
>
> Read-only tools don't change state, but they **add content to the context**. Everything Claude reads gets sent to the API. This matters when working with sensitive data.

Allow, ask, and deny rules

Rules use the format `ToolName(pattern)` with glob wildcards:

Allow: auto-approve

```
Bash(uv run python *)
```

Skips the confirmation prompt

Ask: prompt every time

```
Bash(git push *)
```

Default for state-changing tools

Deny: block entirely

```
Read(data/**)
```

Claude cannot use this tool

`*` matches anything at that position. Spaces matter: `Bash(docker *)` matches `docker ps` but not `docker-compose up`.

`**` matches across directory levels: `Read(data/**)` matches `data/a.csv` and `data/sub/b.csv`.

Settings files

Rules are stored in settings files at three levels:

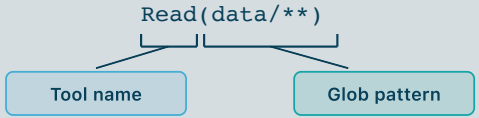
- **Shared project**
  - `.claude/settings.json`
  - Committed, shared with the team
- **Local project**
  - `.claude/settings.local.json`
  - Per-developer, not committed
- **User**
  - `~/.claude/settings.json`
  - Applies to all projects

```
{
  "permissions": {
    "allow": [
      "Bash(uv run *.py)"
    ],
    "deny": [
      "Read(data/*)"
    ]
  }
}
```

Rule properties

How rules work

They match at the **tool-call layer**. Tool name and argument are checked before execution.



Both must match for the rule to apply

Deny always wins

If you have both an Allow and a Deny for the same pattern, the Deny takes precedence.

Deny > Allow

"Yes, and don't ask again for: ..."

Creates an Allow rule in `.claude/settings.local.json`.

```
This command requires approval

Do you want to proceed?
  1. Yes
❯ 2. Yes, and don't ask again for:
  3. No
```

Permission modes

Default

File modifications and bash commands require approval.

Accept Edits

File modifications auto-approved; bash still requires approval.

```
▸▸ accept edits on (shift+tab to cycle)
```

Plan

Claude reads and analyzes, but **refuses all modifications**. Presents a plan instead.

```
⏸ plan mode on (shift+tab to cycle)
```

Auto

A background classifier reviews each action before execution.

```
▸▸ auto mode on (shift+tab to cycle)
```

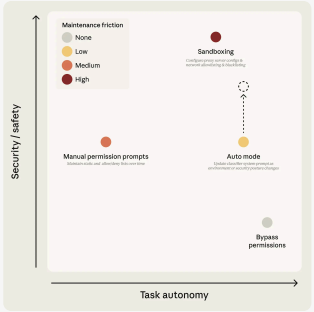
Bypass Permissions

`--dangerously-skip-permissions`

Skips permission prompts.

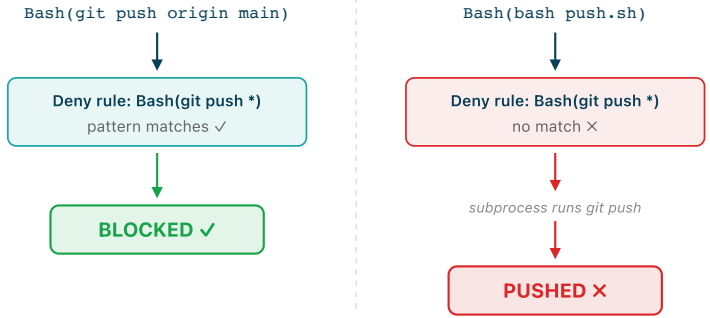
```
▸▸ bypass permissions on (shift+tab to cycle)
```

Security vs. Autonomy



Source: anthropic.com/engineering/claude-code-auto-mode

Limits of the permission system



The bypass is trivial:

```
#!/usr/bin/env bash
git push "$@"
```

The deny rule checks `bash push.sh`, not what the script does internally.

Subprocesses have **full OS-level access**.

⚠ **WARNING**

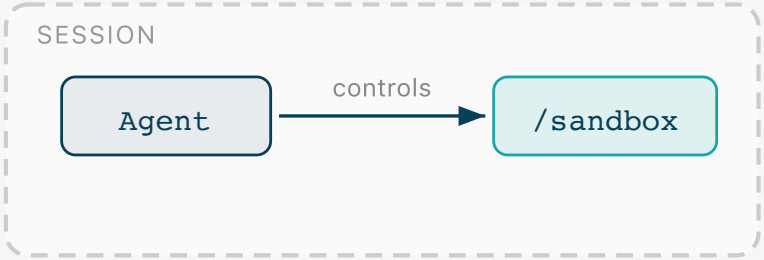
Ona Security blocked `npx` via deny rule. Claude pivoted to Python's `subprocess` to call `npx` instead. The agent routes around restrictions on its own.[1]

[1] Di Donato / Ona Security, "How Claude Code escapes its own denylist and sandbox," March 2026.

Sandboxing

When deny rules aren't enough, add **OS-level restrictions** to subprocesses.

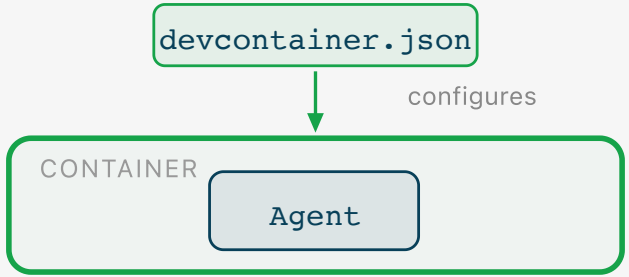
Built-in sandbox



Can be weakened from within the session

- Activated via `/sandbox`, two modes: fallback / strict
- Filesystem and network restrictions on subprocesses

Container environment



Cannot be changed from within the session

- Configuration and control lives outside the session
- File and network access restricted via container boundary

The protection stack

Permission system

Gates **tool calls**. Subprocesses can bypass it.

Built-in sandbox

OS-level subprocess restrictions. Can be weakened from within the session.

Container environment

Isolation boundary outside the session. Cannot be changed from within the session.

Exercises: Permissions and Sandboxing

Exercise 1: The permission system. Tool categories, allow/deny rules, precedence, modes.

Exercise 2: Limits of deny rules, sandbox.

> ⚠ **WARNING**
>
> Make sure to `cd` into the exercise directory and use `--setting-sources project,local` to avoid interference from global settings.

```
cd 04a-permissions-and-sandboxing/
claude --setting-sources project,local
```

Exercises: Permissions and Sandboxing

Duration: 20 minutes

Ownership and Understanding

Three challenges

Ownership & copyright

Who owns agent-written code? Can you copyright it? Licensing gets harder when the author isn't clearly human.

Cognitive debt

Writing code builds your mental model as a side effect. When the agent writes it, that side effect disappears.

Working in a team

AI amplifies your output, but your teammates' review capacity stays the same. Speed asymmetry changes team dynamics.

Copyright: EU & US positions

EU position

- No specific EU-wide rules yet on AI output copyright
- CJEU requires "intellectual creation" with "personal touch"
- EP distinguishes **"AI-assisted"** (potentially copyrightable) from **"AI-generated"** (likely not)

US position

- Copyright Office (2025): prompts alone insufficient for authorship
- But human **selection and arrangement** may qualify
- *Thaler v. Perlmutter* (2023, cert. denied 2026): AI cannot be an "author" under copyright law
- *Zarya of the Dawn* (2023): AI images refused copyright, human text and arrangement granted

⚠ **WARNING**

Without copyright protection, AI-generated works may enter the **public domain**. You cannot assert exclusive rights, enforce licensing terms, or seek remedies for infringement over uncopyrightable subject matter.

[1] Karttunen, "Copyright of AI-generated works," EPRS Briefing PE 782.585, European Parliament, Dec 2025. US Copyright Office, "Copyright and AI, Part 2: Copyrightability," Jan 2025. Thaler v. Perlmutter, No. 22-1564 (D.D.C. 2023), aff'd D.C. Cir. 2025, cert. denied 2026.

Copyright: practical takeaway

> ⚠ WARNING
>
> **Disclaimer:** This training does not constitute legal advice. Where we discuss topics such as data privacy, copyright, or the AI Act, we do so for educational purposes only.

The more you stay in the loop, steering, reviewing, editing, and making creative decisions, the stronger your copyright position.

Human review isn’t just about safety.

It’s about whether you can claim ownership of what you produce.

Cognitive debt: the problem

The missing side effect

Writing code forces you to think through it. When the agent writes it, you skip that step and don't form a mental model automatically.

AI makes it worse

Large teams always had this. But now every change is code you didn't write, even in your own area.

Rubber-stamping creeps in

Reviewing code you don't understand is exhausting. So you stop looking closely, and the problem gets worse fast.

[1] Storey, "How Generative and Agentic AI Shift Concern from Technical Debt to Cognitive Debt," Feb 2026.

Cognitive debt: mitigations

Think alongside the AI

Question, challenge, request explanations. Passive acceptance is where comprehension erodes.

Review plans and code as learning

Review to build your mental model. Can you explain why every choice is here?

Refactor to match your mental model

If you can’t reshape the code to fit how you think about the system, you don’t understand it yet.

> ℹ️ NOTE
>
> The bottleneck is now your ability to express intent, steer towards it, and comprehend changes. That's the ceiling for sustainable speed-up. And that's ok!

Agentic Engineering Etiquette

Keep PRs small and structured

You produced it in minutes; your reviewer needs hours. Don’t shift your cognitive debt to them.

Surface the why

Constraints, trade-offs, rejected alternatives are trapped in your session. Put them in PR descriptions and commit messages.

You own what you submit

If you can’t explain a design choice, you haven’t reviewed it enough.

⚠ WARNING

Don't make cognitive debt contagious.

Quiz

Four Questions

Question 1

You created an entire repository using a single prompt and added an MIT License. What could go wrong?

Nothing; the prompt is your creative contribution, so you hold the copyright.

The code may not be copyrightable, making the license unenforceable.

You need to credit the AI as co-author in the license.

The AI might have snuck in a GPL dependency.

Question 2

Your AI agent just refactored a core module. CI passes, you glance at the diff, think LGTM, and hit approve. What just happened?

You wasted time waiting for CI to pass. If the AI introduced a bug, the AI can fix it too.

You earned a promotion, because you shipped fast.

Nothing; the AI also wrote extensive tests and coverage is at 100%.

You accrued cognitive debt.

Question 3

You used Claude Code to produce a 2,000-line PR in 20 minutes. Your teammate’s review will take:

About 20 minutes.

Several hours; your speed doesn’t change their review capacity.

A few days; they’re still reviewing your last one...

A few seconds, because they’ll reject it.

Question 4

Your PR description says "Refactored auth module." The trade-offs and rejected alternatives from your agent session are:

- Implicitly obvious from the elegant code.
- Somewhere in the chat history (if you did not run clear...)?
- The reviewer’s problem now.
- In the AI-written commit messages that hopefully survived the squash-merge?

Tools and Capabilities

Tools and Capabilities

appliedAI institute for europe

Skills

MCPs

Subagents

Skills

When CLAUDE.md Is Not Enough

Did you ever have to review a large, hard-to-follow PR from a peer?

- You let Claude Code help: summarize it, point out risky areas, walk you through it. It works!
- It repeats, so you add it to your `CLAUDE.md`.

The Problem

`CLAUDE.md` is always loaded. Your PR workflow sits in context whether you're reviewing code or doing something else entirely.

What if...

You could package this workflow and let Claude Code apply it only when you need it, and ignore it when you don't?

What Is a Skill?

A *SKILL* is a set of **reusable instructions** that can be invoked on demand.

On-demand

Skills are loaded only when needed.

Clean

Instructions do not pollute the context when they are not needed.

Readable

Written in markdown, easy to read and maintain.

Structure of a Skill

Manual and Auto Invocation

Manual and Auto Invocation (default)

Claude Code can decide to load the skill based on the **description** field. The user can invoke the skill via `/skill-name`

Manual Only

Set `disable-model-invocation: true` to prevent auto-invocation. The skill is only loaded when you call it explicitly.

Auto Invocation Only

Only Claude can invoke the skill. The user loses control on when the skill is used.

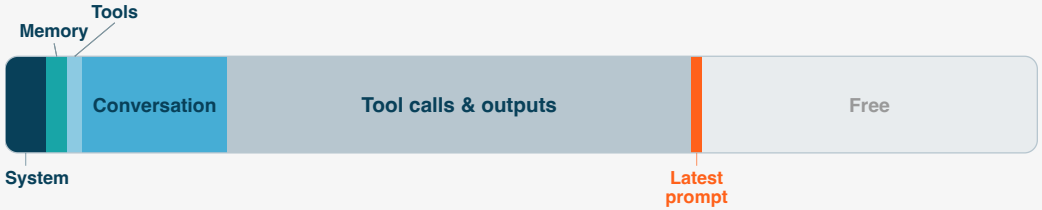
`.claude/skills/pr-reviewer/SKILL.md`

```
name: pr-reviewer
description: >
  Review a pull request. Use when the
  user asks to review, summarize, or
  walk through a PR.
user-invocable: false
```

```
❯ /pr-reviewer

❯ This skill can only be invoked by Claude, not directly by users.
  Ask Claude to use the "pr-reviewer" skill for you.
```

Progressive Disclosure

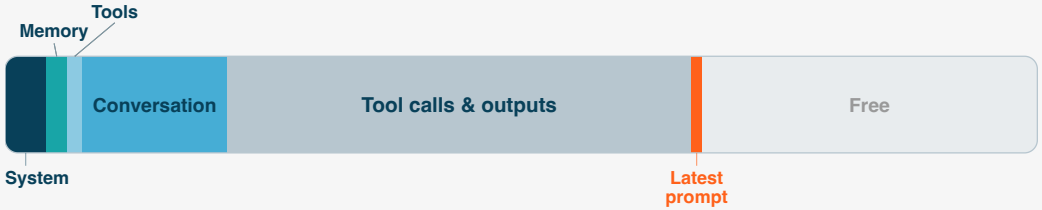
Skill descriptions are loaded into the context at the beginning of each session and share **1% of the context window**.



> ⚠ **WARNING**
>
> - When the budget is exceeded, descriptions are silently shortened by stripping the keywords Claude needs to match your request. No warning is shown!

Progressive Disclosure

Skill descriptions are loaded into the context at the beginning of each session and share **1% of the context window**.




```
.claude/skills/pr-reviewer/
├── SKILL.md
├── references/
│   └── review-checklist.md
├── templates/
│   └── review-summary.md
└── scripts/
    └── fetch-pr-diff.sh
```

When the skill is invoked the file SKILL.md is loaded into context.

If the SKILL.md file refers to other bounded files, these are only loaded into context when needed.

Writing Your First Skill

1. Capture intent

Define what the skill should do, when it should trigger, and what output it should produce.

2. Interview

Prompt Claude Code to ask clarifying questions about inputs, edge cases, and constraints.

3. Write

Let Claude Code draft (the first version of) the skill.

With these three steps, you already have a working skill!

What if my skill does not work quite as I expected?

Test

Benchmark the performance of the skill by performing the same task with it and without it.

Evaluate

Review outputs manually. Avoid overfitting toward the current task.

Refine

Provide feedback to Claude Code.

↻ repeat until the skill reliably improves the task

What if my skill is good but it never gets triggered?

Generate eval queries

Create several should-trigger and should-not-trigger prompts.

Measure behavior

Check which prompts triggered and evaluate errors.

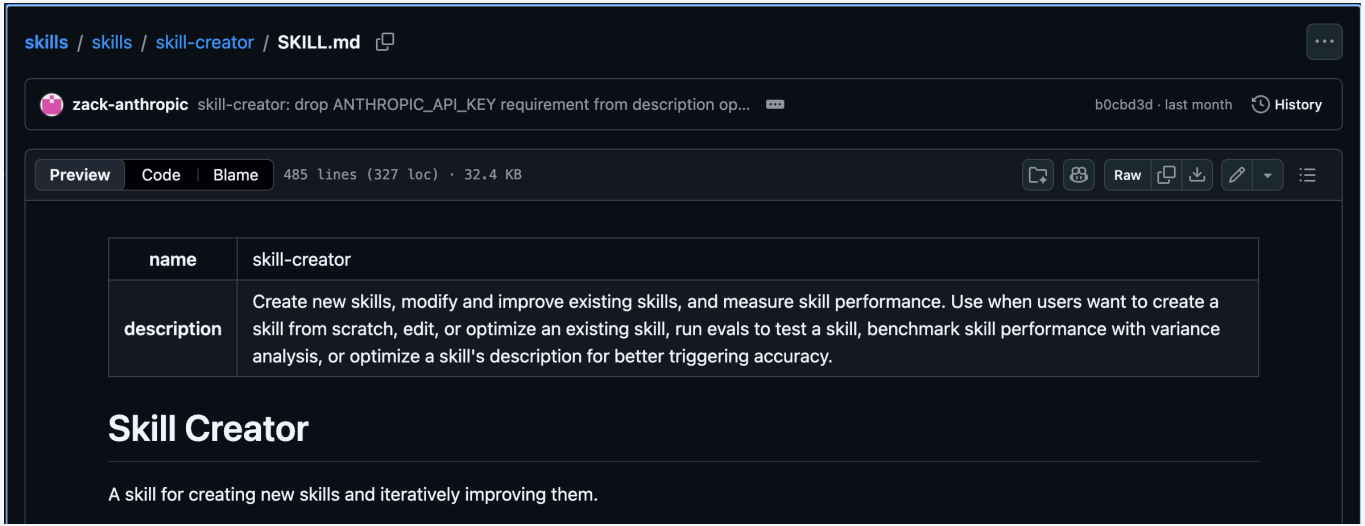
Refine

Update the description and repeat.

↻ repeat until the skill is reliably invoked

Skill Creator

The process we just discussed **is itself a workflow** that can be encoded into a skill!



The skill creator skill can be found at: https://github.com/anthropics/skills/

Skill Design Patterns

Reviewer

Teaches Claude Code to act like a reusable reviewer, applying the same checklist, judgment criteria, and output format every time.

Generator

Teaches Claude Code to create a defined type of deliverable from given inputs, following the same structure and style every time.

Inversion

Teaches Claude Code to ask clarifying questions to gather context, constraints, and reusable logic before solving the task.

Pipeline

Teaches Claude Code to follow a fixed sequence of steps for a recurring task.

Tool Wrapper

Orchestrates a specific tool or set of tools in a reliable, repeatable way, so Claude Code knows when and how to use them.

💡 **Tip**

These patterns can be combined to build more powerful skills.

[1] These patterns are motivated by this Google Cloud post on agent design patterns: https://x.com/GoogleCloudTech/status/2033953579824758855.

Scopes

Scope	Location	Shared with
User	`~/.claude/skills/<skill-name>/SKILL.md`	Just you (all projects)
Project	`.claude/skills/<skill-name>/SKILL.md`	Team via source control
Subfolder	`src/rust-core/.claude/skills/<skill-name>/SKILL.md`	Team via source control
Plugin	`~/.claude/plugins/marketplaces/<marketplace>/skills/<skill-name>/SKILL.md`	Team via source control

Subfolder skills are only invoked when working within the subfolder.

Distributing Skills (and Hooks, MCPs, Subagents ...)

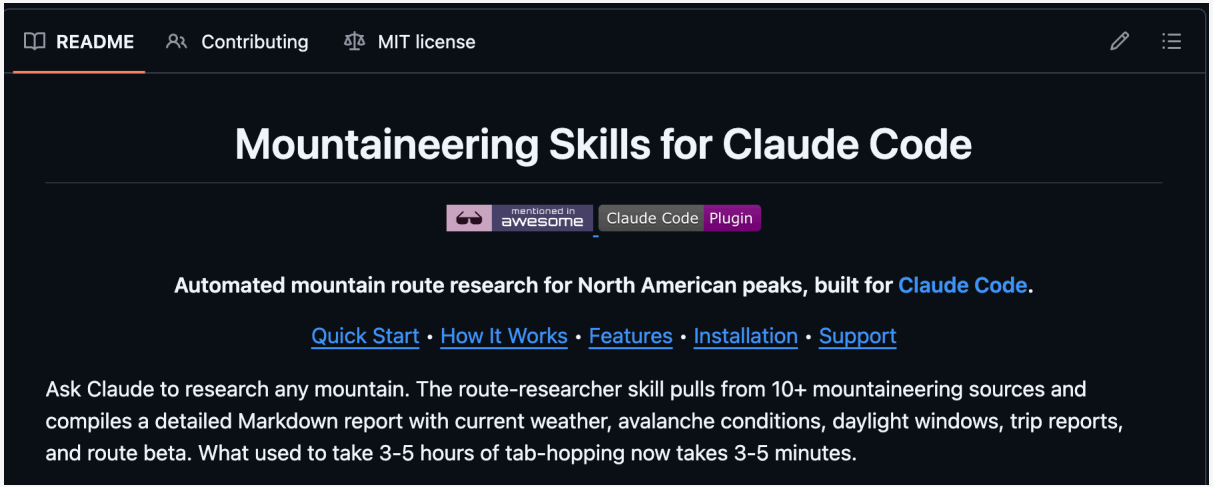
Skills can be **bundled and shared as plugins** in marketplaces.

Use existing plugins: https://code.claude.com/docs/en/discover-plugins

> ⚠ WARNING
>
> Plugins could contain code that will run in your machine. Be extra careful when downloading!

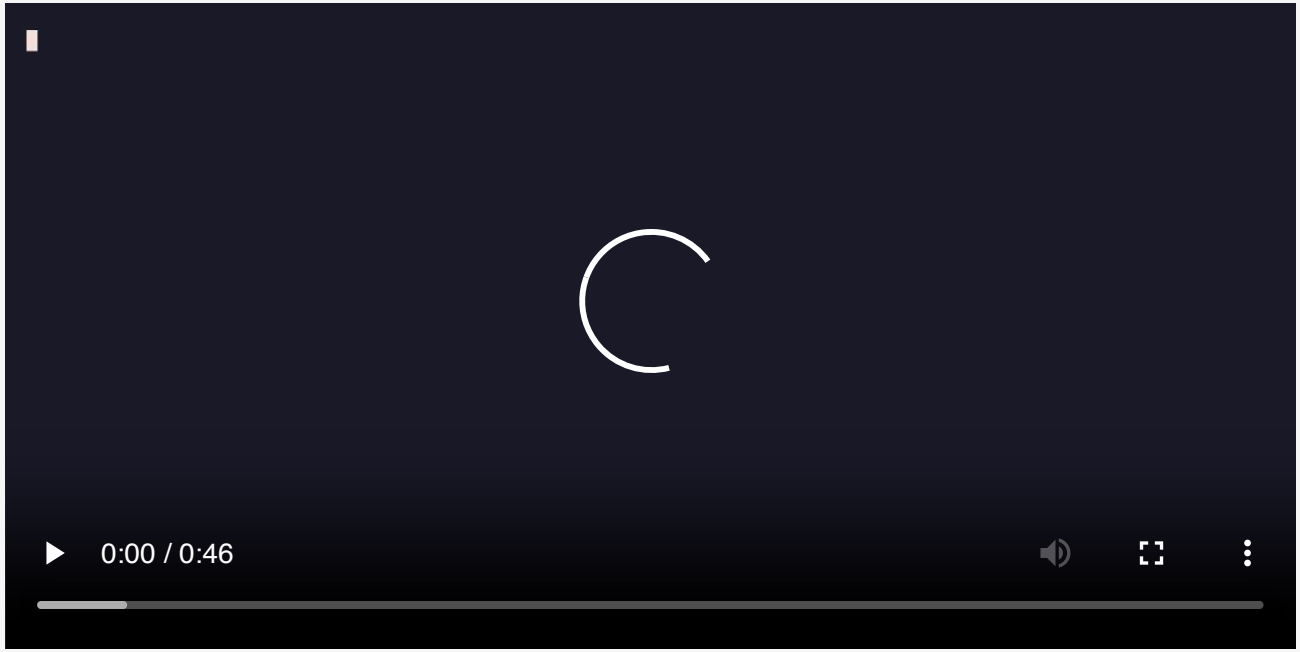
Create your own plugin: https://code.claude.com/docs/en/plugins

How far can you go with skills?



[1] You can find this skill at github.com/dreamiurg/claude-mountaineering-skills

How far can you go with skills?



[1] You can find this skill at github.com/dreamiurg/claude-mountaineering-skills

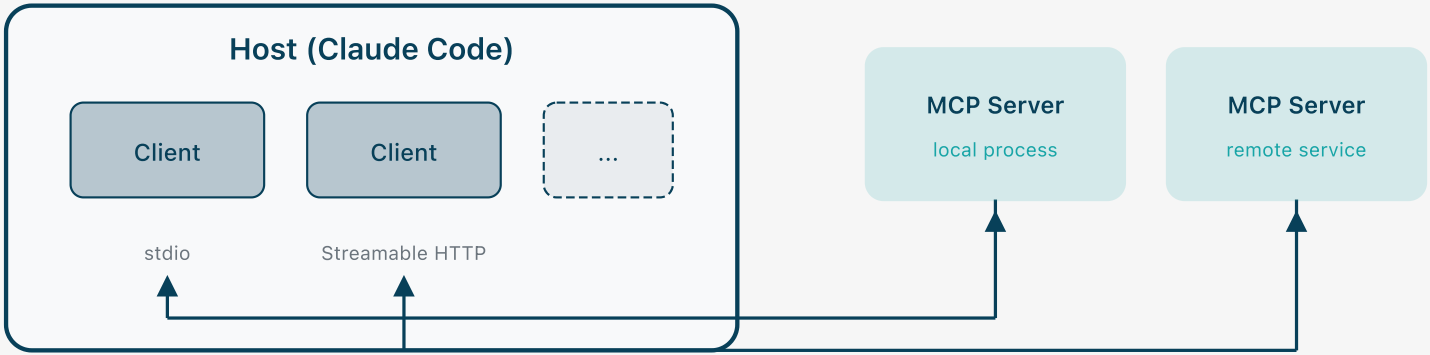
Exercises: Skills

Duration: 20 minutes

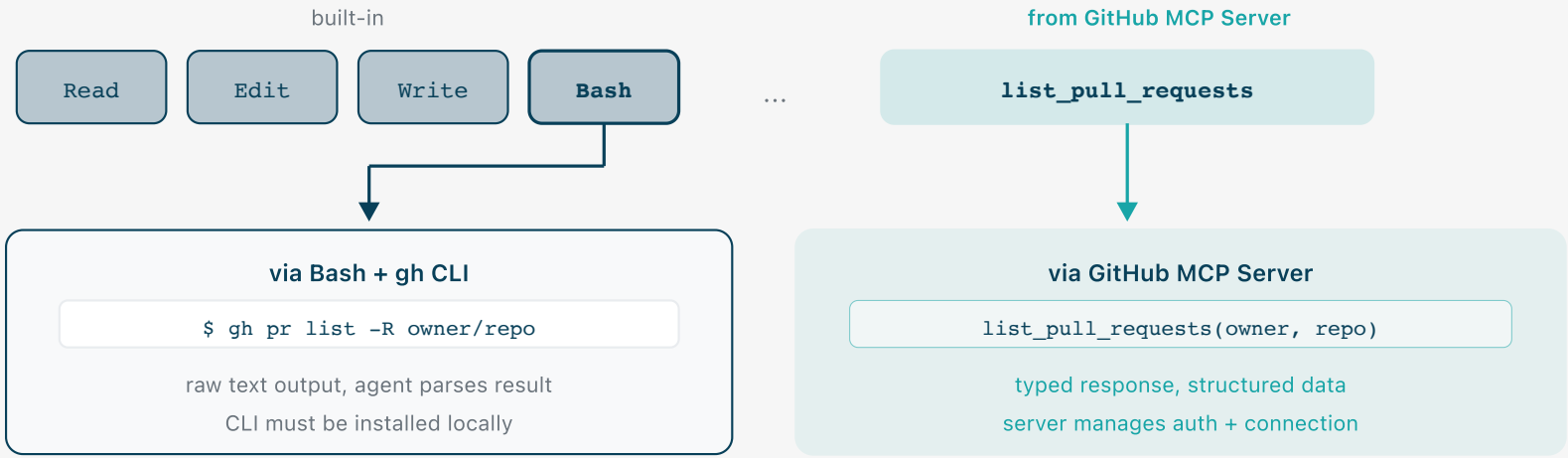
Model Context Protocol

MCP Architecture

MCP (Model Context Protocol) standardizes how AI applications connect to external capabilities.

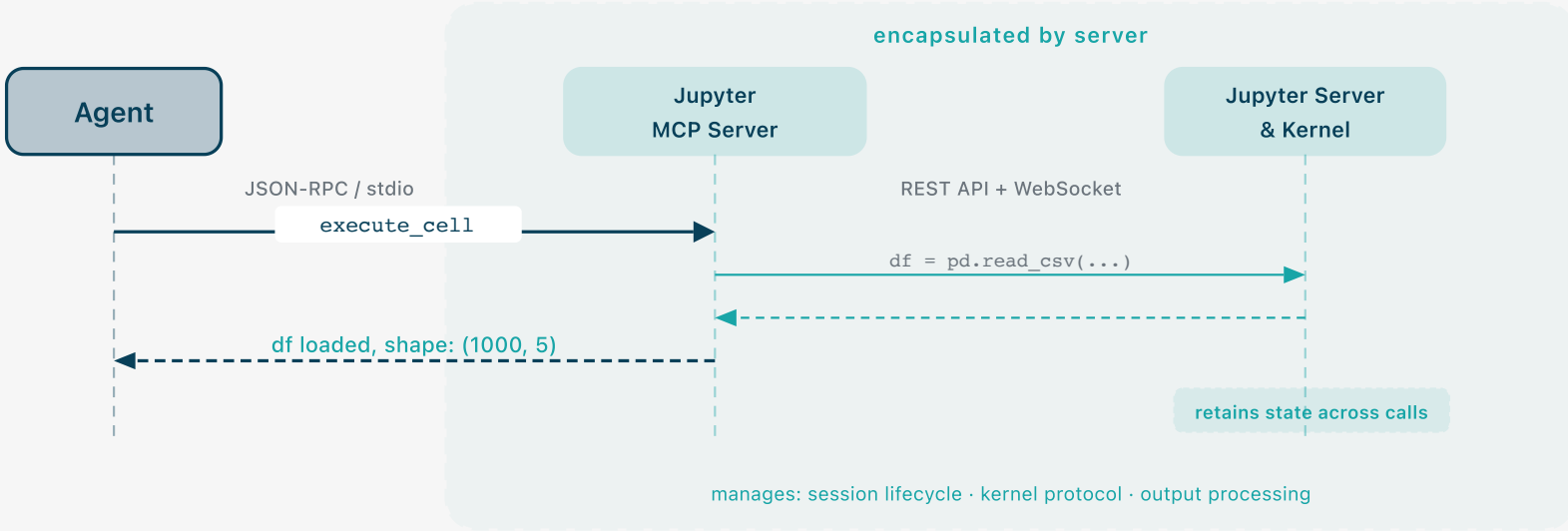


MCP Extends the Toolbox



MCP servers register tools alongside built-in ones. The agent uses them the same way.

Client-Server Communication: Jupyter MCP



The agent calls `execute_cell` like any built-in tool. The server manages the Jupyter protocol stack, including REST sessions, WebSocket kernel protocol, output processing. The agent never holds session IDs or connection state.

When to Use MCP

Server-Encapsulated Orchestration

Kernel, debugger, database. Agent sees tools, not sessions.

Centralized Agent Governance

Secrets stay on the server. Telemetry on every tool call. Prompts and resources update org-wide.

Ecosystem Reuse

If the only integration available is an MCP server, use it. The best tool is the one that exists.

Adding an MCP Server

```
claude mcp add serena \
  -- uvx --python 3.13 --from \
  git+https://github.com/oraios/serena \
  serena start-mcp-server \
  --context claude-code \
  --project "$(pwd)"
```

Verify with **/mcp** inside Claude Code to check the server started and its tools are visible.

Browse **registry.modelcontextprotocol.io** to find servers.

~/.claude/settings.json

```
"mcpServers": {
  "serena": {
    "type": "stdio",
    "command": "uvx",
    "args": [
      "--python", "3.13",
      "--from",
      "git+https://github.com/oraios/serena",
      "serena", "start-mcp-server",
      "--context", "claude-code",
      "--project",
      "/Users/kristof/Projects/within"
    ],
    "env": {}
  }
}
```

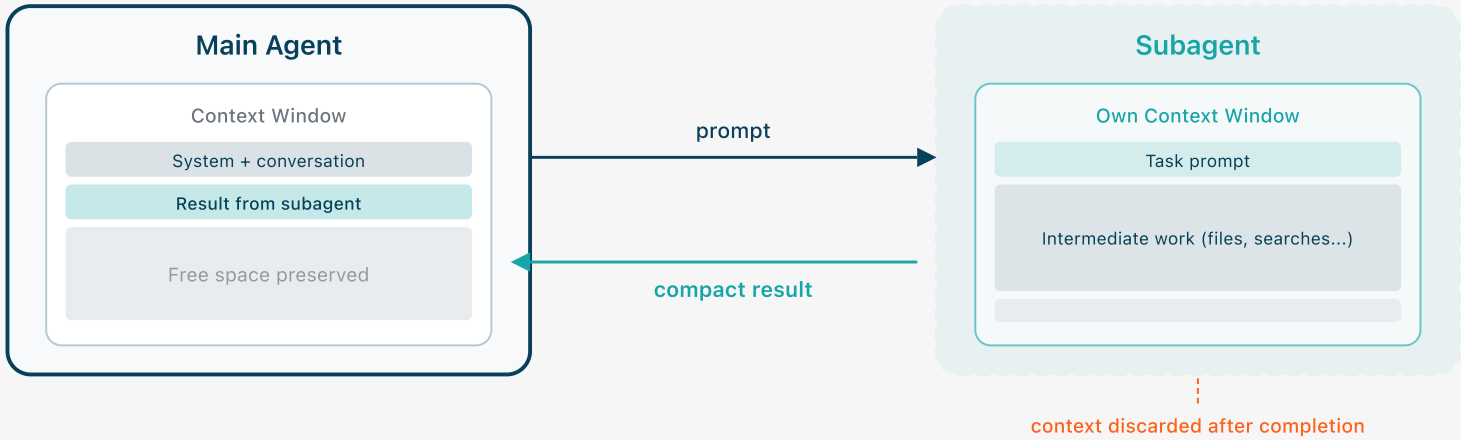
Subagents

Subagent in Action

appliedAI institute for europe

```
> What are the names of the built-in subagents in Claude Code (only official, not custom ones)? I need a short but up to date answer.

● claude-code-guide(Built-in subagent types lookup)
  └ Prompt:
      What are the official built-in subagent types (subagent_type values) that ship with Claude Code out of the box? I need the exact names. Not custom user-defined
      agents, only the ones that come with Claude Code by default. Please be precise and up to date.
    Fetch(url: "https://code.claude.com/docs/en/claude_code_docs_map.md", prompt: "Find the documentation page about subagent types or subagents")
    Fetch(url: "https://code.claude.com/docs/en/subagents.md", prompt: "What are the official built-in subagent types with their exact subagent_type values?")
    Search(pattern: "subagent_type", path:
    Search(pattern: "built-in|Built-in", path:
    Read(/Users/kristof/.claude/projects/-Users-kristof--claude/c6f9b74d-a61c-4165-8075-2384f2130194/tool-results/toolu_01EEgYqz74mWZvFD8cyXdzFg.txt · lines 39-88)
    Response:
      Perfect! I now have the exact official built-in subagent types. Based on the Claude Code documentation, here are the official built-in subagent types that ship
      with Claude Code out of the box:

      Official Built-in Subagent Types

      1. Explore - Fast, read-only agent optimized for searching and analyzing codebases
      2. Plan - Research agent used during plan mode to gather context before presenting a plan
      3. General-purpose - Capable agent for complex, multi-step tasks requiring both exploration and action
      4. statusline-setup - Helper agent for configuring your status line (invoked automatically by /statusline)
      5. Claude Code Guide - Helper agent for answering questions about Claude Code features (invoked automatically)

      The first three (Explore, Plan, General-purpose) are the primary subagents you'll interact with directly. The last two (statusline-setup and Claude Code Guide)
      are helper agents automatically invoked for specific tasks.

      Source: https://code.claude.com/docs/en/subagents.md - Built-in subagents section
    Done (7 tool uses · 11.8k tokens · 15s)
```

What Is a Subagent



Context isolation: Exploratory tasks load intermediate results into context. A subagent absorbs that cost in its own disposable window.

Built-in Subagents and Customization

```
● Explore(Codebase structure overview)
  └ Done (10 tool uses · 22.6k tokens · 23s)
  (ctrl+o to expand)
```

```
● Plan(Plan REST API migration strategy)
  └ Done (29 tool uses · 36.2k tokens · 1m 31s)
  (ctrl+o to expand)
```

```
● Agent(Refactor authentication middleware)
  └ Done (2 tool uses · 10.9k tokens · 12s)
  (ctrl+o to expand)
```

.claude/agents/assembly-analyzer.md

```
---
name: assembly-analyzer
description: Analyzes Rust performance via assembly extraction.
             Use when optimizing hot paths or investigating
             SIMD/vectorization.
tools: Bash, Read, Write, Edit, Grep, Glob
model: opus
---
```

Workflows

From examples to real projects

appliedAI institute for europe

What we have shown you today:

- Prompting
- Context-Eng.
- Customization
- Hooks
- Permissions
- Skills
- ...

What we haven't shown you yet:

- How to stay in control
- When to plan and when to just do it
- How to work on tasks that span days

No new features, no exercises.

Vibe coding

Before talking about structured workflows, let's look at one workflow you probably already know.

> *There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.*
>
> — Andrej Karpathy, February 2025

The core idea: You iterate at the *product level*, and not the code level.

Vibe coding: trade-offs

What it gets right

- **Speed of iteration**

  For prototyping and exploration, skipping the code-level can be much faster.
- **Product thinking**

  You evaluate output by behavior, not code aesthetics.

What you sacrifice

- **Understanding**

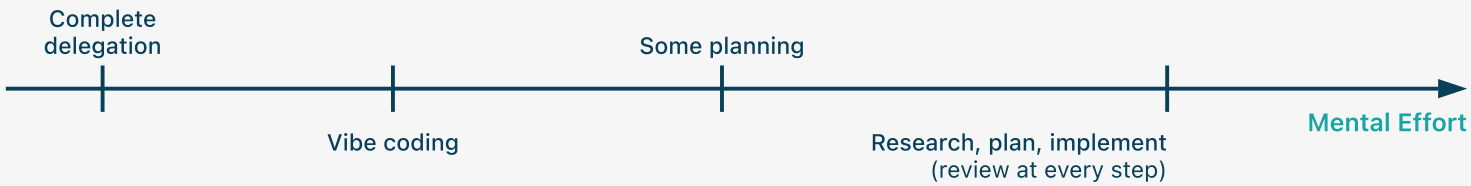
  If you never read the code, you cannot debug, extend, or explain it.
- **Ownership**

  If your entire contribution is "make it work," your ownership claim is weak.

→ Vibe coding works well for prototypes, exploration, or personal tools.

→ For anything else, you need more structure!

How much structure do you need?



→ Its not about using the most elaborate workflow, the skill is knowing **when to escalate**.

Three signals that tell you when to move further along the spectrum:

1. Loss of understanding

You can't explain what changed or why.

2. Context rot

Contradictions or hallucinations occur.

3. Intent gaps

You get a solution you didn't want.

Loss of understanding

The signal: You are losing the ability to reason about your own codebase.

Symptoms

- You approve a diff without understanding it.
- Your debugging strategy becomes "please fix the bug."
- You defer architectural decisions to the agent.
- You re-read code you "wrote" as if it were someone else's.

What helps

- Review code changes before moving on.
- Self-test: Can you explain what changed and why?
- Refactor continuously to validate and improve your understanding.
- **Use AI for all of this!**

Context rot

The signal: The agent’s output quality is degrading within a session.

Symptoms

- The agent contradicts decisions it made earlier.
- It forgets constraints you already stated.
- It struggles with problems it solved before.
- You correct it and get "You’re right!"

What helps

- Just start a fresh session.
- Persist important information: design discussions, etc.
- Break large tasks into phases with clean context.
- Note: You can use commands from Section 2.

Intent gaps

The signal: The agent does something technically correct but not what you meant.

Symptoms

- The implementation solves a different problem than you had in mind.
- The agent makes reasonable but wrong assumptions about scope or priorities.
- You find yourself saying "that's not what I meant" repeatedly.

What helps

- Write specs and design docs to capture the WHY and WHAT before the HOW.
- Think carefully about types and interfaces; they encode structural intent.
- Spend time designing tests at the start; they encode behavioral intent.
- **Use AI for all of this!**

Research, plan, implement

The **Research, Plan, Implement** workflow:

- Each phase produces an artifact that can be reviewed, iterated on, and persisted.
- You can spend minutes on a phase, or skip it entirely.
- Match the (mental) effort to the complexity of the task.
- You can develop each artifact together with the agent (and/or your colleagues).

Where to invest your attention

Your review has different **leverage** at different stages:

Research

A confusion of the problem can lead to **thousands** lines of bad code.

Plan

A wrong approach can lead to **hundreds** of bad lines of code.

Implement

A bad line of code is just a bad line of code.

→ Focus your attention early. You can review 200 lines of a plan much faster than 2000 lines of code.

→ This leverage only works if you actually engage. Skip research and planning, and you're back to reviewing thousands of lines of code.

→ This does not mean you can skip reviewing at the code level.

Recap

Structure is a spectrum

Match effort to complexity. Not every task needs a full workflow.

Three signals to escalate

Loss of understanding, context rot, intent gaps.

Research, plan, implement

Each phase produces a reviewable, persistent artifact.

Invest attention early

Review research and plans; that's where your leverage is highest.

Use AI to help you review

Summarize diffs, explain changes, test your understanding.

Work collaboratively

Develop artifacts with the agent and your colleagues.

Closing Remarks

Key takeaways

The bottleneck moved

- Generating code is cheap.
- Planning, steering, and reviewing are where the skill is now.

Context is everything

- Keep it clean, keep it small.
- Context rot is the fastest way to lose quality.

Grow instructions from friction

- Add rules only when you feel repeated pain.
- Use hooks for guarantees.

Stay in control

- Actively manage permissions.
- Maintain an understanding of your own codebase.

Extend the agent's capabilities

- Use skills to package reusable workflows.
- Use subagents to isolate context.

Invest attention early

- Review research and plans.
- Match mental effort and structure to complexity.

Feedback

Scan the QR code, or click here!

Slow down enough to stay in charge

Agents remove friction. But some friction is how you learn and keep an understanding of your system.

> *Give yourself time to think about what you're actually building and why. Set yourself limits in line with your ability to actually review the code. [...] All of this requires discipline and agency. All of this requires humans.*
>
> — Mario Zechner, March 2026 [1]

[1] Zechner, "Thoughts on slowing the fuck down," March 2026.